LAB 17

Event-Driven Automation: TradingView Webhook to Signal, Risk and Execution

Item	Detail
Topic	Topic 4: Building an End-to-End Algorithmic Trading Pipeline: Backtest, Forward Test and Performance Analysis
Pipeline stage	Event-driven automation
TSC reference	A7, K5, K7
Duration	20 min
Simulator page	http://localhost:8000/#/integrations
Mode	No coding. Runs ONLINE with your Alpaca paper keys (Integrations page), or OFFLINE on the demo data and the built-in paper broker. Paper trading only.

Lion City Capital and every scenario in these materials are fictitious and for training only. All trading in this course is paper (simulated) trading. Nothing in these materials is investment advice or a recommendation to trade any security.

Scenario

Some Lion City Capital traders design signals in TradingView. The desk wants their alerts to place paper orders automatically - but only through the pipeline: a gateway that checks a shared secret, the allow-list, a maximum size and duplicate alerts, asks the pipeline's own signal, runs the risk checks and then hands the order to the order manager, logging every request.

Objective

Send TradingView alerts to the app's webhook and see them pass validation, signal confirmation and risk into paper orders.

Before You Start

- Make sure the Trading Simulator is running and open http://localhost:8000 > Course Labs > Lab 17. It is installed once - see the guide 'Download, Install and Start the Trading Simulator Software' in the Learner Guide.
- Record every result you are asked to check - the same evidence is asked for in the Practical Performance assessment.

What This Lab Is For

In Labs 1-16 you start every run yourself. Many real trading bots are started by an event instead - here a TradingView alert. When your strategy fires on a TradingView chart, TradingView sends a short message (JSON) to a web address called a webhook, and the simulator turns that message into a paper order automatically. You do not get a webhook from TradingView: the simulator gives you the address and you paste it into the TradingView alert. Because anyone who finds that address could send it a message, the simulator acts as a gateway and checks every alert before it trades: Is it from us (the secret)? Is it allowed (the symbol allow-list and maximum size)? Is it readable (valid JSON)? Have we seen it already

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

(duplicate)? Optionally, does the app's own signal agree? Only then do the risk checks run and the paper order go in, and every request is written to the audit log. The lesson: automation is only safe when every incoming signal is checked before any order is placed.

Lab Steps - In the Trading Simulator

No coding is needed. Make sure the Trading Simulator is running (installed once - see the guide 'Download, Install and Start the Trading Simulator Software') and open http://localhost:8000. In the sidebar open Course Labs > Lab 17: the app shows the same steps with a Go button that opens each page pre-filled. Every step works offline on the bundled demo data and the built-in paper broker.

Step 1. Copy the alert template

Open Integrations and scroll down to the TradingView webhook card. It holds the two things TradingView needs: where to send an alert (the Webhook URL) and what to send (the Alert message). Read each part below - you paste both into TradingView in Step 3.

- **Webhook URL:** the simulator's address for incoming alerts, ending in /api/webhooks/tradingview. As shown (http://localhost:8000/...) only your own computer can reach it; Step 3 explains how to make it public.
- **Alert message:** the JSON that TradingView sends with every alert. TradingView replaces each {{...}} placeholder when the alert fires: {{ticker}} = the symbol, {{strategy.order.action}} = buy or sell, {{strategy.order.contracts}} = the quantity, {{close}} = the price, {{timenow}} = the time (part of the alert id).
- **secret:** a password shared by TradingView and the simulator, so only your alerts are accepted. It is shown masked; press Reveal secret only on your own machine and never share it, paste it into a chat or show it on a slide.
- **Allowed symbols, Max qty per alert:** the gateway's own limits: an alert for any other symbol, or a larger quantity, is rejected before any order
- **Reject alerts that disagree with the app's own signal:** optional signal confirmation: the app re-checks each alert against the strategy in 'Confirm with strategy' and rejects a disagreeing alert (HTTP 409). Leave it unticked for this lab.

Open: http://localhost:8000/#/integrations (Integrations > TradingView webhook)

Check: You can say what each part of the message is for and which values TradingView fills in (the {{...}} placeholders).

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 2. Run the self-test

Press 'Run self-test (7 simulated alerts)'. The app sends itself seven alerts exactly as TradingView would - two good ones and five that each break one rule - so you can watch every safeguard work without a TradingView account or the internet. The HTTP code is the answer the simulator sends back to TradingView: 200 = accepted, 4xx = rejected.

- **1-2. Valid BUY and SELL 5 SPY:** HTTP 200 order submitted - they passed every check and became paper orders
- **3. Wrong secret:** HTTP 401 - not from you: someone who finds your URL cannot trade
- **4. Symbol not allowed (TSLA):** HTTP 422 - not on the allow-list
- **5. Qty above the limit (5,000 AAPL):** HTTP 422 - a typo or runaway alert is stopped by the maximum size
- **6. Malformed message:** HTTP 400 - not valid JSON, e.g. a broken Message box in TradingView
- **7. TradingView retry of alert 1:** HTTP 200 duplicate ignored - TradingView resends an alert if the reply is slow; the same alert id is processed only once, so you never buy twice

Open: http://localhost:8000/#/integrations

Check: 7/7 simulated alerts handled as expected: valid buy and sell (200), wrong secret (401), symbol not allowed and qty too large (422), malformed (400), retry ignored (200 duplicate). The Recent alerts table (the audit log, secret redacted) lists every one.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 3. Real alerts (paid TradingView plan)

Optional - trainer demonstration, or try it yourself if you have a paid TradingView plan. This is the same flow with a real TradingView alert. TradingView's servers are on the internet: they cannot reach localhost on your laptop and only post to public HTTPS addresses (port 80/443). So the simulator first gets a public address from a tunnel (ngrok), then the alert is created on a TradingView chart with that address and the message from Step 1.

- **You need:** a paid TradingView plan (webhooks are not on the free plan) with two-factor authentication switched on (Profile > Settings > Privacy and Security); a free ngrok account and ngrok installed (ngrok.com); the simulator running
- **1. Start the tunnel:** in a terminal run: ngrok http 8000. Copy the Forwarding address, e.g. https://ab12-34-56-78-90.ngrok-free.app. Keep the window open - closing it ends the tunnel. On the free ngrok plan the address changes every time ngrok restarts, so paste the new one into the alert.
- **2. Your webhook URL:** the Forwarding address + /api/webhooks/tradingview, e.g. https://ab12-34-56-78-90.ngrok-free.app/api/webhooks/tradingview. Test: open the Forwarding address in a browser - after ngrok's 'Visit Site' notice the simulator loads (TradingView does not see the notice).
- **3. Open a chart:** tradingview.com > Products > Supercharts (not the home page). Pick a symbol on the allow-list, e.g. SPY or KO.
- **4. Add the strategy:** Pine Editor (bottom of the chart) > paste the Pine Script v6 exported from the simulator's Strategy Library (as in Lab 5) > Save > Add to chart. Its buy and sell markers appear on the chart.
- **5. Open the alert dialog:** the Alert button (alarm clock with +) in the top toolbar, or Option+A (Alt+A on Windows). Not the bell icon in the right sidebar - that is Community notifications.
- **6. Condition:** change Price to your strategy's name; keep 'Order fills only'
- **7. Message:** click the Message row (it reads e.g. 'SPY Crossing 770.00 >'), delete the text and paste the Alert message from Step 1 with the secret revealed - a masked secret is rejected with HTTP 401
- **8. Notifications:** click the 'App, Toasts >' text on the Notifications row, tick Webhook URL, paste your webhook URL and go back - the row now includes Webhook. If Webhook URL is locked, the plan does not include webhooks.
- **9. Create:** press Create: the alert fires the next time the strategy places an order
- **Quick demo with a Price alert:** the {{strategy...}} placeholders are filled only for a strategy alert. For a 'Price Crossing' alert (fires within minutes if the value is next to the current price) write the action and quantity into the message: {"secret": "<revealed secret>", "id": "{{timenow}}", "ticker": "{{ticker}}", "action": "buy", "qty": 1, "price": "{{close}}"}
- **Watch it arrive:** the ngrok window's Connections count goes up, and http://127.0.0.1:4040 shows the request and the simulator's reply (200 / 401 / 422); then Integrations > Recent alerts and Paper Trading > Orders (source webhook)

Open: http://localhost:8000/#/integrations

Check: The alert appears in the audit log and its order in Paper Trading > Orders (source webhook).

Expected in the app: 7 simulated alerts handled as expected and the audit log.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

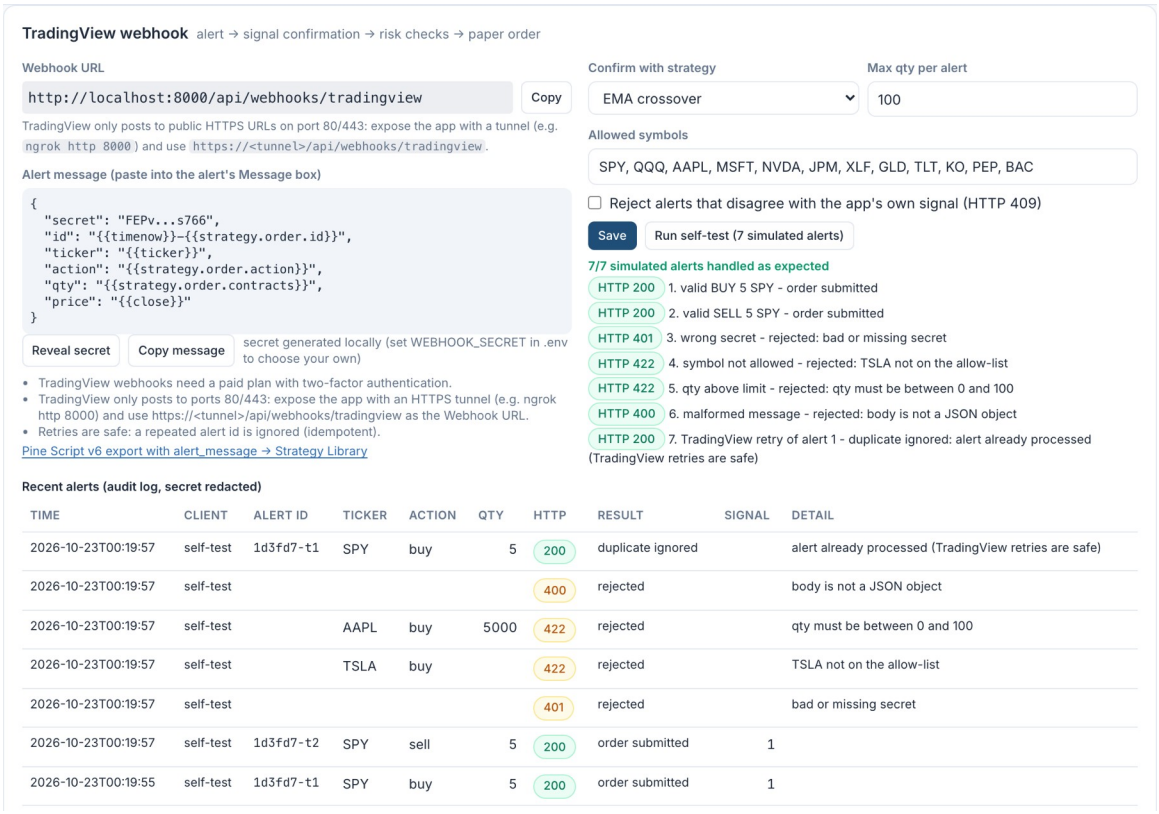


Figure: Lab 17 in the Trading Simulator.

Optional: Let AI Write the Code

This lab needs no coding. To see, run or customise the code behind it, click a link: the AI opens with the prompt filled in and writes the code for you. All prompts are also in labs_lowcode/Prompt_Library.html and .pdf.

A webhook that turns TradingView alerts into paper orders - run it in Google Colab
I am a beginner and do not write code. Write ONE complete Python script that runs as-is in a Google Colab notebook. Start with a !pip install line for any library Colab lacks, use pandas, numpy, matplotlib, scipy, scikit-learn, statsmodels and yfinance only, get daily prices from yfinance (auto_adjust=True unless I say otherwise), put a one-line comment above each section, print results as clear tables and charts, and after the code explain in 5 short bullet points how to read the output. Paper trading and education only. Task: write a small FastAPI app with a /webhook endpoint that receives a TradingView alert JSON {secret, ticker, action, price}, rejects a wrong secret, validates the fields, confirms the signal with a 20-day SMA check, caps the position size, and logs the paper order it would send. Test it inside Colab with FastAPI's TestClient by posting 7 sample alerts (valid buy, valid sell, wrong secret, missing field, unknown ticker, duplicate, oversized) and print each result.

Open in Claude | Open in ChatGPT | Open Google Colab

Debrief

A webhook lets a charting platform trigger execution on a trading platform (K5), but the gateway code (A7) is where the controls live: authentication, allow-lists, size limits, duplicate protection, the pipeline's own signal, the risk layer and an audit log. Each automated order still pays spread and slippage (K7), so alert frequency matters.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.